Control Flow: Loops (CodeGrade)

 (https://github.com/learn-co-curriculum/phase-3-python-loops)  (https://github.com/learn-co-curriculum/phase-3-python-loops/issues/new)

Learning Goals

- Write basic loops with the `for` and `while` constructs.
- Use generator and list comprehensions to accomplish complex tasks quickly.

Key Vocab

- **Interpreter**: a program that executes other programs. Python programs require the Python interpreter to be installed on your computer so that they can be run.
- **Python Shell**: an interactive interpreter that can be accessed from the command line.
- **Data Type**: a specific kind of data. The Python interpreter uses these types to determine which actions can be performed on different data items.
- **Exception**: a type of error that can be predicted and handled without causing a program to crash.
- **Code Block**: a collection of code that is interpreted together. Python groups code blocks by indentation level.
- **Function**: a named code block that performs a sequence of actions when it is called.
- **Scope**: the area in your program where a specific variable can be called.

Introduction

In this lesson, we'll show how to use control flow to run the same line(s) of code multiple times in a loop. Make sure to follow along by opening the Python shell and experimenting with the example code.

Basic Loops in Python

In JavaScript, there are a few common approaches to control flow that will allow us to run the same lines of code over and over again. The most basic tool is the `while` loop, which works like this in JavaScript:

```
let i = 0;
while (i < 5) {
  console.log("Looping!");
  i++;
}
```

Python has a `while` construct too, which works in much the same way:

```
i = 0
while i < 5:
  print("Looping!")
  i += 1
```

Looping with `for`

JavaScript has a `for` loop, which is commonly used to run some condition a set number of times:

```
for (let i = 0; i < 10; i++) {
  console.log(`Looping!`);
  console.log(`i is: ${i}`);
}
```

Python also has a `for` loop (with slightly simpler syntax!):

```
for i in range(10):
    print("Looping!")
    print(f"i is: {i}")
```

When writing a `for` loop in Python, the loop can proceed through any iterable object type. These include `list`, `tuple`, `set`, and `dict` objects, as well as `str` and `range` objects. `range` objects are especially useful in the `for` construct because they generate an ordered sequence of `int`s from 0 through a number of your choosing (like 10, above).

A `for` loop in Python automatically proceeds to the next element of the iterable object in its constructor; there is no need to specify that the loop is stopping at a certain point or increasing after each iteration.

List Comprehensions

Imagine, if you will, that you are an analyst who has been hired to predict how college athletes would perform after transitioning to the NBA. One of the important metrics you're taking into consideration is *height*.

You meticulously measured each player in the NCAA, but now that you have all of the data in front of you, you can see that you've made a horrible mistake. You measured all of the heights in **furlongs (https://www.britannica.com/science/furlong)**.

```
player_heights = [0.008, 0.008, 0.008, 0.009, 0.008, 0.01, 0.009, 0.009, 0.01, 0.008, 0.009, 0.009, 0.008, 0.008, 0.
```

We could certainly write a `for` loop to handle this:

```
inch_heights = list()
for height in player_heights:
    inch_height = height * 7920
    inch_heights.append(inch_height)
```

...but now we've written four lines of code when we only want to do a simple conversion.

List comprehensions allow us to transform sequences of data in a single line. Here's how we would accomplish the above task with a list comprehension:

```
inch_heights = [height * 7920 for height in player_heights]
```

That's it! Another benefit of list comprehensions is that you can easily reuse the name of your original sequence without worrying about conflicting names:

```
player_heights = [height * 7920 for height in player_heights]
```

Now it's almost like your mistake never happened at all.

```
> print(player_heights)
# [63.36, 63.36, 63.36, 71.28, 63.36, 79.2, 71.28, 71.28, 79.2, 63.36, 71.28, 71.28, 63.36, 63.36, 63.36, 71.28, 63.
```

List comprehensions are a very powerful tool, but there are jobs that a `for` loop is better suited for. There are two main factors to keep in mind when choosing between the two:

1. List comprehensions should only be used for loops where the output is an iterable object such as a `list` or `set`.
2. `for` loops separate steps into different lines, which is how human eyes expect to see instructions. List comprehensions are restricted to a single line and can be difficult for other humans to understand.

Instructions

Time to get some practice! Write your code in the `looping.py` file. Run `pytest -x` to check your work. Your goal is to practice using control flow in Python to familiarize yourself with the syntax. There is a JavaScript version of the solution for each of these deliverables in the `js/index.js` file you can look at (but if you want an extra challenge, try solving them in Python without looking at the JavaScript solution).

Write a function `happy_new_year()` using a `while` loop that outputs numbers starting at 10 and counting down to 1. After reaching 1, print out "Happy New Year!"

```
happy_new_year
# 10
# 9
# ...
# 2
# 1
# Happy New Year!
```

Write a function `square_integers()` that takes one argument, a list of integers and returns the list of squared elements.

```
square_integers([1, 2, 3, 4, 5])
# [1, 4, 9, 16, 25]
```

Write a function `fizzbuzz()` that prints the numbers from 1 to 100. For multiples of three, print "Fizz" instead of the number and for the multiples of five print "Buzz". For numbers which are multiples of both three and five, print "FizzBuzz".

```
fizzbuzz()
# 1
# 2
# Fizz
# 4
# Buzz
# Fizz
# 7
# ...
# 14
# FizzBuzz
# 16
# ...
```

```
# 98
# Fizz
# Buzz
```

Resources

- **Python For Loops (https://wiki.python.org/moin/ForLoop)**
- **Python While Loops (https://wiki.python.org/moin/WhileLoop)**
- **List Comprehension vs. For Loop (https://www.programiz.com/python-programming/list-comprehension)**

This tool needs to be loaded in a new browser window

Load Control Flow: Loops (CodeGrade) in a new window